Éditeur de documents partagé

Dossier de conception

Écriture collaborative en temps réel
et distribution de documents à quota strict

sur une infrastructure AWS

Auteur Djusse Tameno Christian

Filière 5e année Génie informatique, ENSPY

Cours Cloud — projet d'évaluation individuel

Région AWS `eu-north-1` (Stockholm)

Dépôt `github.com/Djusse//Editeur_Partag-`

Ce dossier décrit la conception telle qu'elle est *implémentée*, et non telle qu'elle avait été prévue. Les écarts sont signalés, avec leur raison.
Chaque affirmation sur les contraintes renvoie à un test ou à un scénario de charge exécutable.

Table des matières

1 Introduction **4**
1.1 Objet du document . . . 4
1.2 Ce que ce dossier est, et n'est pas . . . 4
1.3 Plan . . . 4

2 Contexte, objectif et contraintes **4**
2.1 Contexte . . . 4
2.2 Les six contraintes . . . 5
2.3 Hypothèses d'interprétation . . . 5
2.4 Objectif mesurable . . . 6

3 Partie A — Architecture globale **6**
3.1 Diagramme de contexte . . . 6
3.2 Architecture de déploiement AWS . . . 7
3.3 Décisions d'architecture et alternatives écartées . . . 9
3.4 Flux internes . . . 9
3.4.1 Réservation d'un exemplaire . . . 10
3.4.2 Écriture groupée . . . 10
3.5 Sécurité et bonnes pratiques . . . 11
3.6 Infrastructure, déploiement et suppression . . . 12
3.7 Limites et risques . . . 13

4 Partie B — Conception de l'application **13**
4.1 Cas d'utilisation . . . 13
4.2 Diagramme de paquets . . . 14
4.3 Diagramme de classes et modèle de données . . . 15
4.4 Diagrammes d'états-transitions . . . 17
4.4.1 États du lien . . . 17
4.4.2 États du visiteur . . . 18
4.4.3 États du crédit . . . 19
4.5 Diagrammes de séquence système . . . 19
4.5.1 Réserver et télécharger . . . 20
4.5.2 Éditer un document . . . 21
4.5.3 Restaurer une version . . . 22
4.6 Conception par contrainte . . . 22
4.6.1 C1 — exactement N téléchargements . . . 23
4.6.2 C2 — aucune modification acceptée perdue . . . 23
4.6.3 C3 — document tel qu'il est à T_1 . . . 23
4.6.4 C4 — jamais plus que son crédit . . . 24
4.6.5 C5 — 15 000 clics et 300 rédacteurs . . . 24

4.6.6 C6 — dernier exemplaire : prévenir vite, arrêter le travail 25

5 Conclusion **25**

5.1 Ce qui est tenu, et comment on le sait 25

5.2 Ce qui reste à mesurer 26

5.3 Ce que ce projet a réellement appris 26

5.4 Perspectives 26

5.5 Mot de la fin 26

Table des figures

1 Diagramme de contexte : trois acteurs, un système, deux services extérieurs. 7
2 Architecture de déploiement : une région, une instance applicative, quatre services gérés. 8
3 Flux de réservation : transaction conditionnelle, puis génération asynchrone du PDF. 10
4 Flux d'écriture : un lot par tick de 250 ms, une transaction atomique, un seul message diffusé. 11
5 Cas d'utilisation : rédacteur, visiteur, administrateur. 14
6 Organisation des paquets du backend. 15
7 Modèle du domaine : dix entités, trois portent une règle de concurrence ou de quota. 16
8 États d'un lien de téléchargement. 17
9 États d'un visiteur sur un lien. 18
10 États du crédit d'écriture, par jour et par adresse. 19
11 Séquence système : réserver puis télécharger un exemplaire. 20
12 Séquence système : éditer un document en collaboration. 21
13 Séquence système : restaurer une version. 22

Liste des tableaux

1 Les six contraintes et la réponse architecturale retenue. 5
2 Hypothèses d'interprétation, à confronter à l'énoncé. 5
3 Décisions d'architecture et alternatives écartées. 9
4 Mesures de sécurité et bonne pratique correspondante. 12
5 Limites connues et risques associés. 13
6 Modèle de données : dix tables, une clé et un rôle par table. 17
7 Synthèse : une contrainte, un mécanisme, une preuve. 25

1 Introduction

1.1 Objet du document

Ce dossier décrit la conception d'un éditeur de documents partagé déployé sur AWS. L'application poursuit deux buts que l'énoncé oppose volontairement :

— plusieurs centaines de personnes écrivent *en même temps* dans le même texte, et aucune frappe acceptée ne doit être perdue ;

— un lien public distribue le document en PDF à *exactement* N personnes, jamais $N+1$, chacune recevant le document tel qu'il était au moment où elle a réservé son exemplaire.

Le premier objectif demande de la souplesse : des écritures concurrentes, un historique, de la reprise après coupure. Le second demande de la rigidité : un compteur exact, une transaction, une réponse unique. Tenir les deux dans la même application est l'essentiel du travail de conception.

1.2 Ce que ce dossier est, et n'est pas

Ce dossier décrit la conception *réellement implémentée*, vérifiée contre le code source, et non une intention de conception. Trois conséquences :

1. là où l'implémentation s'écarte du plan initial, l'écart est signalé avec sa raison — ces écarts sont souvent la partie la plus instructive ;
2. chaque affirmation sur une contrainte renvoie à un test unitaire, un script d'intégration ou un scénario de charge, tous exécutables ;
3. les limites sont énoncées sans être atténuées. Une limite annoncée vaut mieux qu'une limite découverte par le lecteur.

1.3 Plan

La **partie A** traite de l'architecture : contexte, déploiement, décisions et alternatives écartées, flux internes, sécurité, infrastructure, limites.

La **partie B** traite de la conception applicative : cas d'utilisation, organisation des paquets, modèle du domaine, états, séquences système, puis une lecture par contrainte, de C1 à C6.

La **partie C** conclut : ce qui est prouvé, ce qui reste à mesurer, et ce que ce projet a réellement appris.

2 Contexte, objectif et contraintes

2.1 Contexte

Le sujet impose de démontrer la maîtrise des services AWS au travers d'une application non triviale, dont chaque choix doit pouvoir être justifié. Il précise que « une application simple, mais dont chaque choix est compris et justifié, vaut mieux qu'une application complexe que vous ne savez pas expliquer ». Ce dossier suit cette consigne : peu de services, mais chacun employé pour ce qu'il fait de mieux, et chaque fois avec l'alternative écartée.

Deux points du sujet sont éliminatoires : les contraintes C1 et C2. Une application rapide qui les viole est non fonctionnelle, quel que soit son polissage.

2.2 Les six contraintes

#	Contrainte de l'énoncé	Réponse retenue
C1	Exactement N téléchargements, jamais $N+1$	transaction DynamoDB : décrément de `remaining` et création du ticket dans la même écriture conditionnelle. Une garde en mémoire allège la base sous le pic, mais n'est jamais l'autorité
C2	Aucune modification acceptée perdue ; convergence	CRDT Yjs avec *un seul écrivain* par document ; une modification n'est acquittée qu'*après* écriture durable dans le journal
C3	Document tel qu'il est à T_1, complet et cohérent	le claim fige un numéro de séquence T_1 ; le PDF est construit depuis le journal jusqu'à ce numéro, une seule fois par version, de façon asynchrone
C4	Jamais plus que son crédit, même sur plusieurs appareils	compteur DynamoDB par (e-mail, jour) débité dans la même transaction que l'écriture au journal ; comptage effectué côté serveur, sur le binaire Yjs
C5	15 000 clics (80 % en 60 s) pendant que 300 rédacteurs écrivent	micro-cache nginx sur l'état du lien, garde en mémoire sur les réservations, diffusion par lots de 250 ms, instance dimensionnée par la mesure. Trois états explicites : prêt, patienter, épuisé
C6	Dernier exemplaire : visiteurs prévenus vite, plus de travail inutile	état épuisé monotone : réponse immédiate, aucun appel à la base, mis en cache longuement par nginx

Table 1 – Les six contraintes et la réponse architecturale retenue.

2.3 Hypothèses d'interprétation

Le sujet laisse plusieurs points ouverts. Les choix ci-dessous ne sont pas confirmés par l'enseignant : ils sont donc explicités, pour que le lecteur puisse contester l'interprétation plutôt que le résultat.

Sujet	Interprétation retenue
Consommation d'un exemplaire	au clic qui réserve le téléchargement, et non au téléchargement effectif du fichier
Instant T_1	lecture du dernier numéro de séquence au début du claim
Crédit	10 000 caractères par jour ; insertions *et* suppressions comptées ; restauration de version gratuite
Recharge du crédit	minuit, heure de Douala — et non minuit UTC (voir B.6)
Blocage	4 refus pour crédit épuisé dans la journée bloquent l'écriture jusqu'à la recharge ; la lecture et le téléchargement restent possibles
Pic d'écriture	300 rédacteurs dans le *même* document : c'est le pire cas, retenu comme cible ; le palier réellement atteint est celui qui est rapporté
Identité	nom et e-mail, sans mot de passe. Le sujet l'autorise ; l'e-mail est la clé du crédit et du partage
Livrables	français, dépôt GitHub privé, jury ajouté en collaborateur

Table 2 – Hypothèses d'interprétation, à confronter à l'énoncé.

Hypothèse d'interprétation. La recharge du crédit à minuit *heure de Douala* s'écarte de l'hypothèse initiale (minuit UTC). L'écart vient de l'usage : un utilisateur à Douala lisait « recharge à 01 :00 » et ne comprenait pas. Le décalage est constant toute l'année, le Cameroun n'appliquant pas d'heure d'été, ce qui rend l'arithmétique exacte sans table de fuseaux.

2.4 Objectif mesurable

La conception n'est acceptable que si elle est mesurable. Quatre jalons ont guidé les décisions :

1. N réservations exactement pour un lien de N exemplaires, sous 15 000 clics dont 80 % en 60 secondes, sans aucune erreur serveur ;
2. zéro modification acceptée sans acquittement, sous écriture simultanée de plusieurs dizaines de rédacteurs ;
3. convergence du contenu vérifiable : journal sans trou, reconstruction identique par deux chemins indépendants ;
4. un palier de rédacteurs réellement atteint, rapporté tel quel, même s'il est inférieur à la cible.

Le troisième point mérite d'être souligné : un test de charge mesure un *débit d'acquittement*, pas une convergence de contenu. Les deux preuves sont distinctes, et le dossier les traite séparément.

3 Partie A — Architecture globale

3.1 Diagramme de contexte

Le système a trois catégories d'utilisateurs, qui n'accèdent pas au service de la même façon.

Le **rédacteur** s'identifie par un nom et une adresse électronique, sans mot de passe, et reçoit un jeton de session. Il crée des documents, les partage, les versionne et fabrique des liens de téléchargement.

Le **visiteur** est le cas intéressant : il ne s'identifie *jamais*. Le jeton du lien qu'il détient *est* sa capacité d'accès. Cette décision a une conséquence architecturale : le parcours visiteur est exposé à la racine (`/d/{token}/...`), en dehors du préfixe `/api` qui regroupe les routes authentifiées. La confondre avec les autres routes produit un échec silencieux : la requête aboutit sur le fichier statique du front et renvoie un code 200 avec du HTML.

L'**administrateur** consulte l'état du système avec une clé dédiée, dans un en-tête, et non avec le jeton de session : c'est une capacité d'exploitation, pas une identité applicative.

Deux éléments extérieurs complètent le tableau. **Amazon SES** envoie un unique courriel lorsque le dernier exemplaire d'un lien est distribué. **Le stockage objet** sert les fichiers PDF : le système ne les transporte jamais, il présigne une adresse et laisse S3 répondre. C'est ce qui permet à 500 téléchargements de ne pas passer par l'instance applicative.

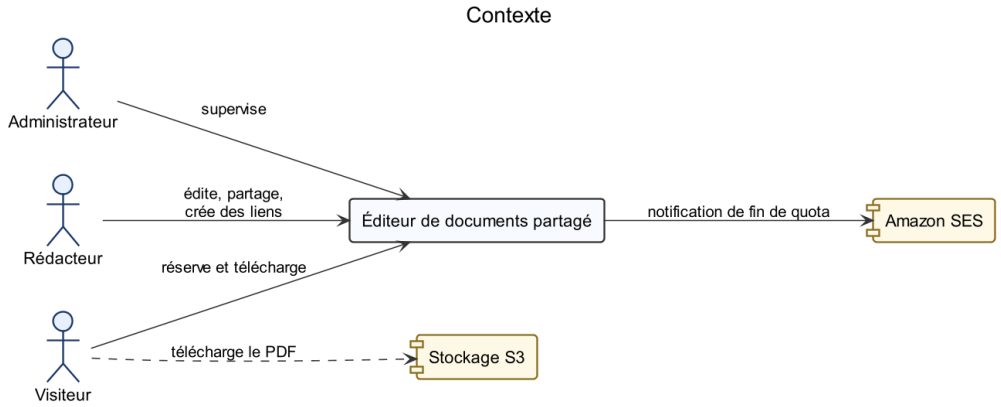


Figure 1 – Diagramme de contexte : trois acteurs, un système, deux services extérieurs.

Un point de conception mérite d'être explicité ici. Le destinataire du courriel de fin de quota n'est pas un acteur du système : il ne dispose d'aucune interface, ne peut rien demander et ne reçoit rien d'autre. Le représenter comme un acteur donnerait une idée fausse du périmètre.

3.2 Architecture de déploiement AWS

Tout le trafic entre par une **adresse IP élastique**, attachée à une instance unique. Cette adresse est indispensable : une adresse publique ordinaire change à chaque arrêt puis démarrage de l'instance, ce qui rendrait l'évaluation impossible avec une URL stable.

Sur l'instance cohabitent trois rôles, séparés par des processus distincts et non par des machines :

— **nginx** sert le front statique, met en micro-cache l'état des liens, et relaie vers Node les préfixes `/api`, `/d` et `/ws` ;

— **le processus applicatif** tient l'état Yjs des documents ouverts en mémoire et sert l'API REST et le WebSocket ;

— **le processus de travail** consomme la file de génération de PDF, séparément pour qu'un document lourd ne bloque pas les écritures.

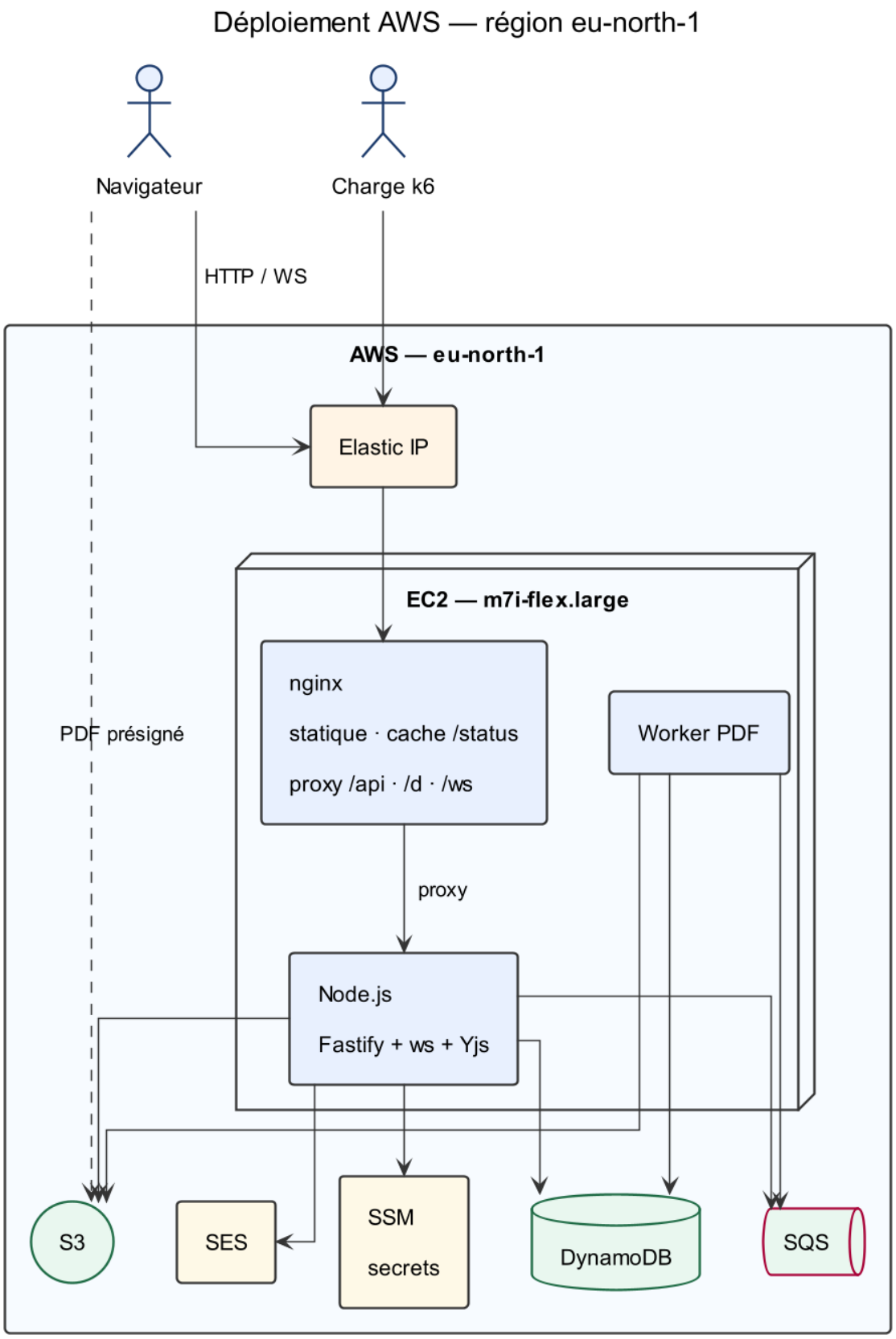


Figure 2 – Architecture de déploiement : une région, une instance applicative, quatre services gérés.

Le choix du type d'instance est une décision de dimensionnement, pas de confort. Le type retenu est `m7i-flex.large` : deux processeurs virtuels et huit gigaoctets de mémoire. Deux raisons, dans cet ordre :

1. c'est le seul type à la fois **éligible à l'offre gratuite** et doté de la mémoire nécessaire pour tenir des milliers de connexions simultanées. `t3.large` a été essayé d'abord : la création de pile échoue avec « not eligible for Free Tier » ;
2. les instances de la famille `t` consomment des crédits de processeur et se retrouvent bridées quand ces crédits s'épuisent. En mode *Standard*, une instance bridée s'effondre précisément pendant le pic qu'on cherche à mesurer. Le mode *illimité* évite le bridage mais rend la facture imprévisible.

3.3 Décisions d'architecture et alternatives écartées

Chaque décision est consignée dans un document d'architecture séparé (`docs/adr/`), avec ses alternatives. Le tableau ci-dessous en donne la synthèse et, surtout, ce qui a été *écarté* : une décision sans alternative n'est pas une décision.

Choix	Retenu, et pourquoi	Écarté, et pourquoi
Calcul	**Instance EC2 unique** : le WebSocket et l'état en mémoire sont naturels, et un seul écrivain par document rend C2 simple à tenir	**Lambda + API Gateway** : la limite de 400 exécutions simultanées du compte est trop serrée pour 15 000 clics, et le WebSocket persistant obligerait à externaliser tout l'état
Convergence	**CRDT Yjs** : les modifications concurrentes fusionnent sans verrou, et un client hors ligne se réconcilie à la reconnexion	**Verrou exclusif** : un seul rédacteur à la fois, ce que le sujet interdit ; **transformation opérationnelle** : juste, mais coûteux à écrire et à prouver dans le temps d'un projet
Écriture	**Écrivain unique et lots de 250 ms** : une transaction par lot au lieu d'une par frappe	**Écriture par frappe** : une transaction DynamoDB par caractère épuise le débit disponible et multiplie la facture
Persistance	**DynamoDB** : transactions conditionnelles, ce qui donne C1 exactement, et aucune administration de serveur	**PostgreSQL** : les transactions seraient plus expressives, mais imposent un serveur à administrer, sauvegarder et dimensionner
Fichiers	**S3 privé + URL présignée de 60 s** : le fichier ne traverse pas l'instance	**Flux servi par Node** : 500 PDF simultanés saturent la bande passante de l'instance et bloquent l'édition
PDF	**Génération asynchrone par file** : le visiteur reçoit « patienter » et revient ; un seul travail par version grâce à un verrou	**Génération dans la requête** : une requête de plusieurs secondes sous 15 000 clics ferait tomber l'instance
Pic visiteur	**Micro-cache nginx sur l'état du lien** : une seconde de cache absorbe 15 000 lectures sans toucher la base	**Interrogation directe de DynamoDB** : 15 000 lectures par seconde la font échouer en limitation de débit
Front	**Export statique servi par nginx** : quelques fichiers, aucun serveur applicatif pour le rendu	**Rendu côté serveur Next.js** : un second processus Node à dimensionner, sans bénéfice ici

Table 3 – Décisions d'architecture et alternatives écartées.

3.4 Flux internes

Deux flux méritent une figure : celui qui tient la contrainte C1, et celui qui tient C2 avec C4. Ce sont les deux contraintes éliminatoires.

3.4.1 Réservation d'un exemplaire

L'ordre des vérifications est imposé par le sujet et a des conséquences observables. Avant de décider que le lien est épuisé, le serveur cherche *d'abord* si ce visiteur possède déjà un ticket. Sans cette précaution, un visiteur déjà servi recevrait « épuisé » après l'épuisement du lien — il perdrait un exemplaire qu'il avait légitimement obtenu.

Ensuite, la réservation est une transaction conditionnelle : décrémenter `remaining` *si* il est supérieur à zéro et si le lien est actif, et créer le ticket *si* ce visiteur n'en a pas. La transition vers l'état épuisé est une écriture séparée, elle aussi conditionnelle : seul le claim qui amène réellement le compteur à zéro peut la poser. C'est cette séparation qui rend l'épuisement idempotent lorsque cinquante visiteurs arrivent ensemble.

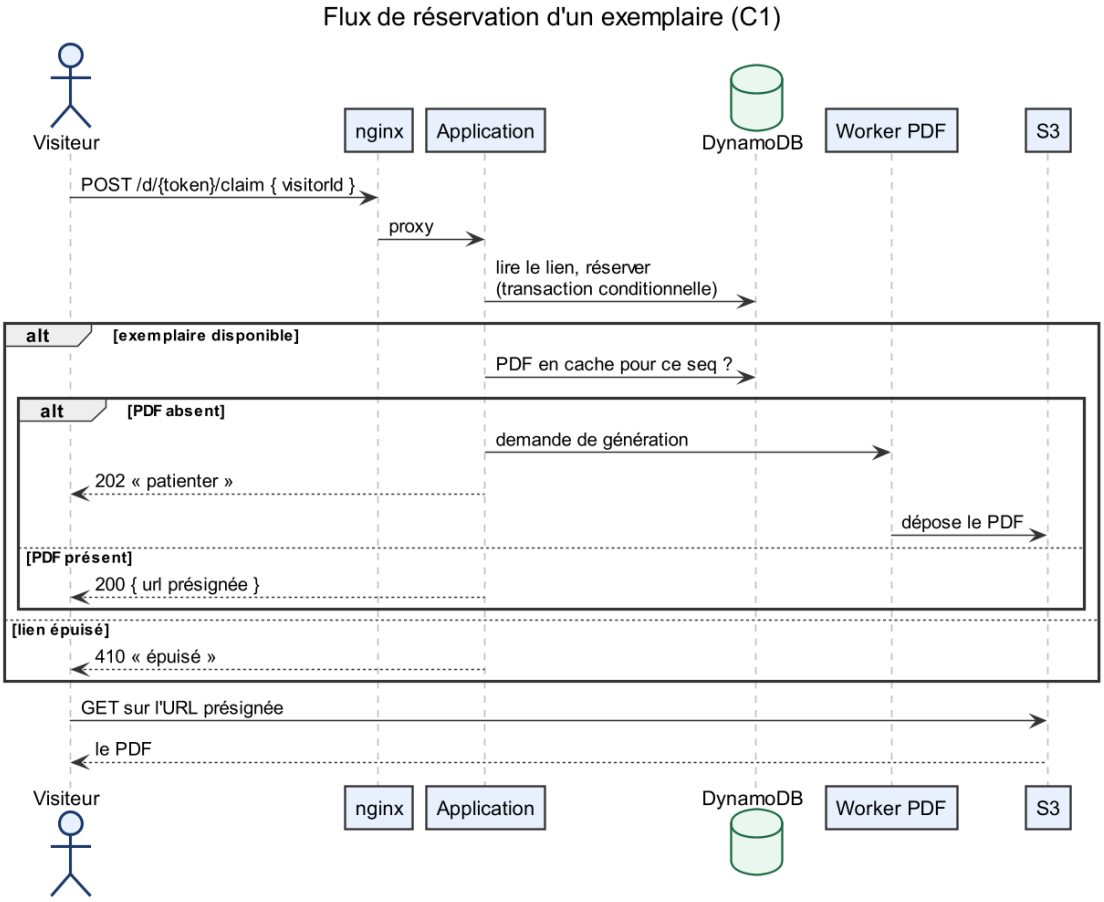


Figure 3 – Flux de réservation : transaction conditionnelle, puis génération asynchrone du PDF.

Le numéro de séquence est figé au moment du claim, avant la transaction. Tous les visiteurs d'une même rafale partagent donc le même fichier, et une modification ultérieure du document ne change pas leur exemplaire.

3.4.2 Écriture groupée

Le flux d'écriture porte deux contraintes à la fois : aucune modification perdue (C2) et aucun dépassement de crédit (C4). Les deux se règlent au même endroit, parce qu'elles portent sur la même transaction.

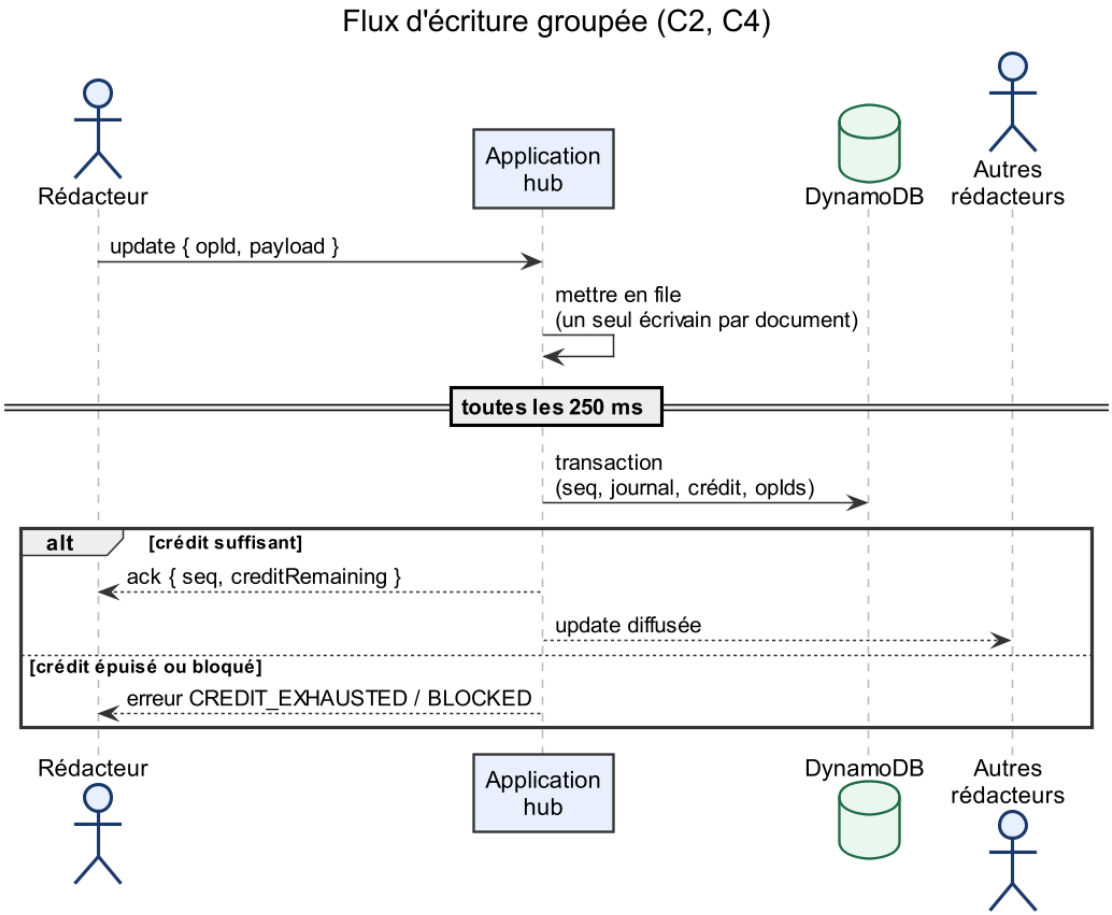


Figure 4 – Flux d'écriture : un lot par tick de 250 ms, une transaction atomique, un seul message diffusé.

Un point de conception a coûté cher et mérite d'être rapporté. Le contrôle du crédit ne peut pas être une condition de la transaction : DynamoDB refuse l'arithmétique entre deux valeurs dans une expression de condition. La forme `used + :chars <= :limit` est rejetée par le service. Le contrôle se fait donc par une lecture préalable, hors transaction. Le risque de course est borné par l'écrivain unique ; la transaction reste la garde d'atomicité pour la séquence, le journal et les marqueurs d'idempotence.

3.5 Sécurité et bonnes pratiques

Les mesures ci-dessous répondent directement aux bonnes pratiques du cours.

Point	Mise en œuvre
Aucune clé d'accès dans le code	le poste utilise une connexion fédérée ; le serveur utilise le rôle de l'instance, donc des identifiants temporaires renouvelés
Aucun secret dans les données de démarrage	les données de démarrage d'une instance sont lisibles via le service de métadonnées. Les deux secrets (signature des sessions, clé d'administration) sont dans le magasin de paramètres, créés une fois et conservés
Moindre privilège	le rôle de l'instance n'accède qu'aux tables nommées, au préfixe de stockage, à la file, et à l'envoi de courriel filtré sur l'expéditeur vérifié
Métadonnées d'instance	jeton obligatoire : sans session, les données de démarrage ne sont pas lisibles
Stockage objet	accès public bloqué, chiffrement au repos, versionnage, règle refusant le trafic non chiffré, URL présignées de 60 secondes
Pare-feu	port 80 ouvert ; port 22 *fermé* par défaut, accès par session sécurisée. Une valeur permissive est explicitement traitée comme une fermeture, et non comme une ouverture
Jeton WebSocket	le navigateur ne permet pas d'en-tête d'authentification au handshake : un ticket à usage unique de 60 secondes évite d'exposer le jeton de session dans l'URL, donc dans les journaux du serveur frontal
Validation	adresse électronique, longueur du titre, longueur de l'identifiant d'opération, plafond de 64 Kio par mise à jour, corps de requête limité

Table 4 – Mesures de sécurité et bonne pratique correspondante.

3.6 Infrastructure, déploiement et suppression

L'infrastructure entière est décrite dans un seul modèle de pile, ce qui rend le déploiement reproductible : une instance, une adresse élastique, un groupe de sécurité, dix tables, un compartiment de stockage, une file avec file de lettres mortes, un rôle d'instance et un groupe de journaux.

Le script de déploiement enchaîne six étapes, et chacune échoue franchement :

1. vérification de l'identité, de la région, et de l'*éligibilité* du type d'instance — interrogée auprès du service, et non tenue dans une liste à la main ;
2. création éventuelle de la paire de clés, puis des deux secrets ;
3. déploiement de la pile, avec suppression préalable si elle se trouve dans un état d'échec antérieur, qui interdit toute mise à jour ;
4. construction du backend puis du front, en libérant les dépendances au fur et à mesure pour tenir dans l'espace disque disponible ;
5. envoi de l'archive vers le stockage, puis installation sur l'instance par commande à distance ;
6. vérification que le service répond, que le parcours visiteur est relayé, et que le micro-cache est actif.

La suppression mérite un script dédié, parce qu'une simple suppression de pile échoue : le compartiment est versionné, donc il faut purger les objets *et* leurs versions ; les paramètres et la paire de clés vivent hors de la pile et survivent à sa suppression. Le script purge, supprime, puis *vérifie* huit familles de ressources, en distinguant trois états : absent, restant, et *inconnu* — une commande de contrôle qui échoue ne doit jamais être comptée comme « rien ne reste ».

3.7 Limites et risques

Limite	Conséquence, et raison du choix
Trafic sans chiffrement	les jetons et les adresses circulent en clair. Un certificat exige un nom de domaine, absent du sujet. Limite assumée, documentée, et non dissimulée
Instance unique	aucune haute disponibilité : un arrêt de la machine interrompt le service. C'est le prix d'un écrivain unique par document, qui simplifie C2. Le redémarrage automatique ramène le processus, pas la machine
Identité sans mot de passe	quiconque connaît une adresse peut écrire en son nom. Le sujet l'autorise ; c'est aussi ce qui rend l'identité du visiteur inutile
Jeton non révocable	un jeton volé reste valable jusqu'à son expiration. Sans liste de révocation, il n'y a pas de contournement autre que le renouvellement
Rattrapage plafonné	la lecture du journal s'arrête à un nombre d'entrées fixé, sans signaler la troncature : un client doit boucler. Un appel unique sur un document ancien donnerait un journal partiel, donc une vérification de convergence fausse
Restauration non facturée	une restauration de version est gratuite et franchit un blocage. C'est un recours voulu, mais le compteur ne reflète donc pas tout le travail effectué
Bac à sable du courriel	tant que l'accès de production n'est pas accordé, seule l'adresse vérifiée reçoit la notification

Table 5 – Limites connues et risques associés.

Hypothèse d'interprétation. L'absence de haute disponibilité est présentée comme une limite assumée, non comme un oubli. Le cours demande explicitement de savoir et de dire qu'une instance unique n'est pas disponible. Les pistes d'évolution — plusieurs instances avec élection d'un écrivain par document, ou externalisation de l'état — sont plus coûteuses que le bénéfice attendu pour un projet évalué sur la justification des choix.

4 Partie B — Conception de l'application

4.1 Cas d'utilisation

Trois acteurs, onze cas. La lecture utile n'est pas la liste mais les deux relations entre cas, qui portent chacune une décision.

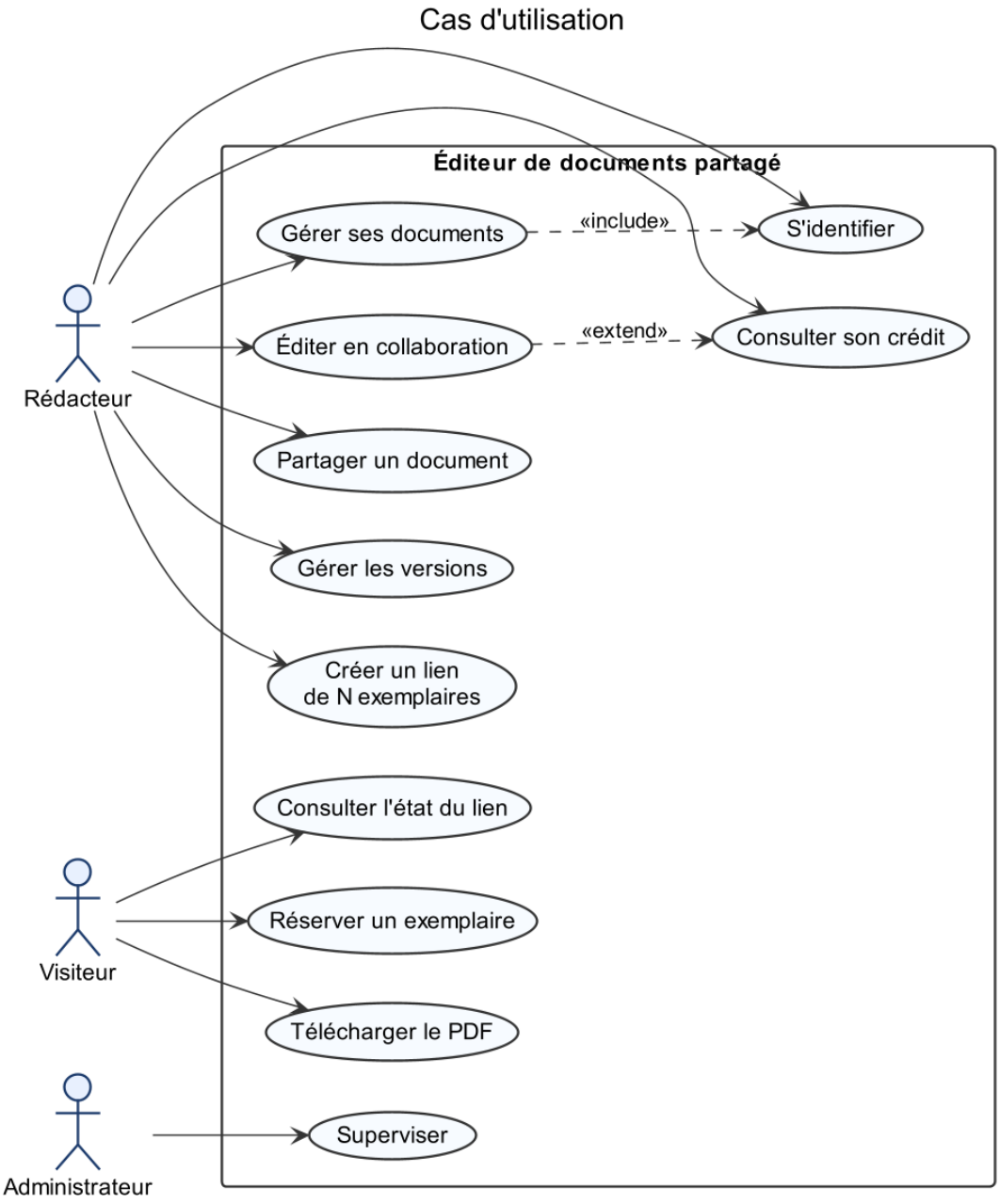


Figure 5 – Cas d'utilisation : rédacteur, visiteur, administrateur.

La première relation paraît anodine et ne l'est pas : « éditer en collaboration » *étend* « consulter son crédit », et non l'inverse. Le crédit n'est jamais interrogé avant d'écrire. Le retour du serveur — acquittement ou refus — porte déjà le crédit restant, et c'est le refus qui met la jauge à jour. Une relation d'inclusion aurait décrit un aller-retour qui n'existe pas.

La seconde est une absence qu'il faut assumer : il n'y a *aucune* relation entre « réserver un exemplaire » et « s'identifier ». Le visiteur ne s'identifie jamais. Ce n'est pas un oubli dans la conception, c'est la décision qui rend le parcours visiteur absorbable à 15 000 clics : pas de session à créer, pas de jeton à vérifier, une seule écriture conditionnelle.

4.2 Diagramme de paquets

L'organisation du code suit une règle unique : les routes dépendent du domaine, le domaine dépend des accès aux services, jamais l'inverse. Un seul paquet parle aux services AWS. Le domaine reçoit le stockage, la file et le courriel par injection, ce qui permet aux tests de n'appeler aucun service.

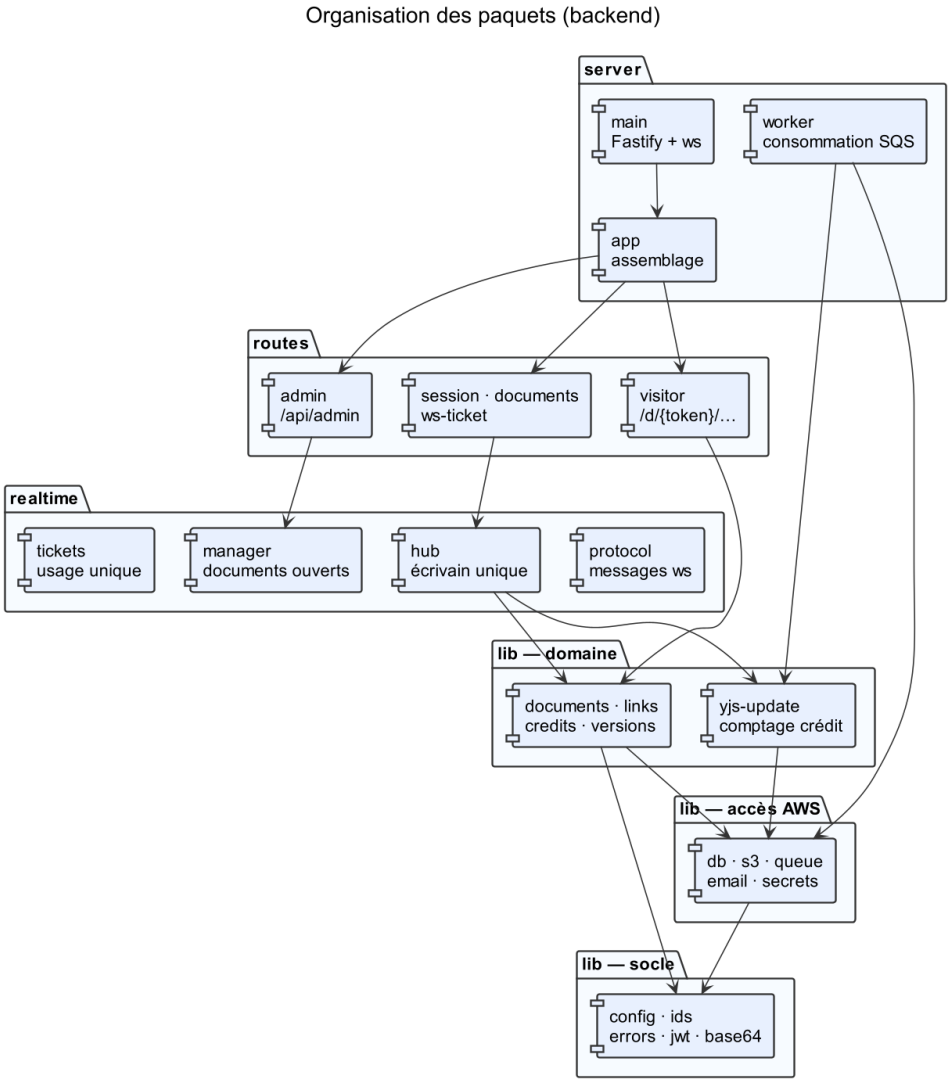


Figure 6 – Organisation des paquets du backend.

Cette organisation n'est pas décorative. C'est elle qui a permis d'écrire 43 tests sans aucun appel réseau, et c'est elle qui a rendu possible le mode local complet, où l'application tourne avec un stockage et une base en mémoire, sans compte AWS.

4.3 Diagramme de classes et modèle de données

Le modèle tient en dix tables, dont trois portent une règle plutôt qu'une donnée.

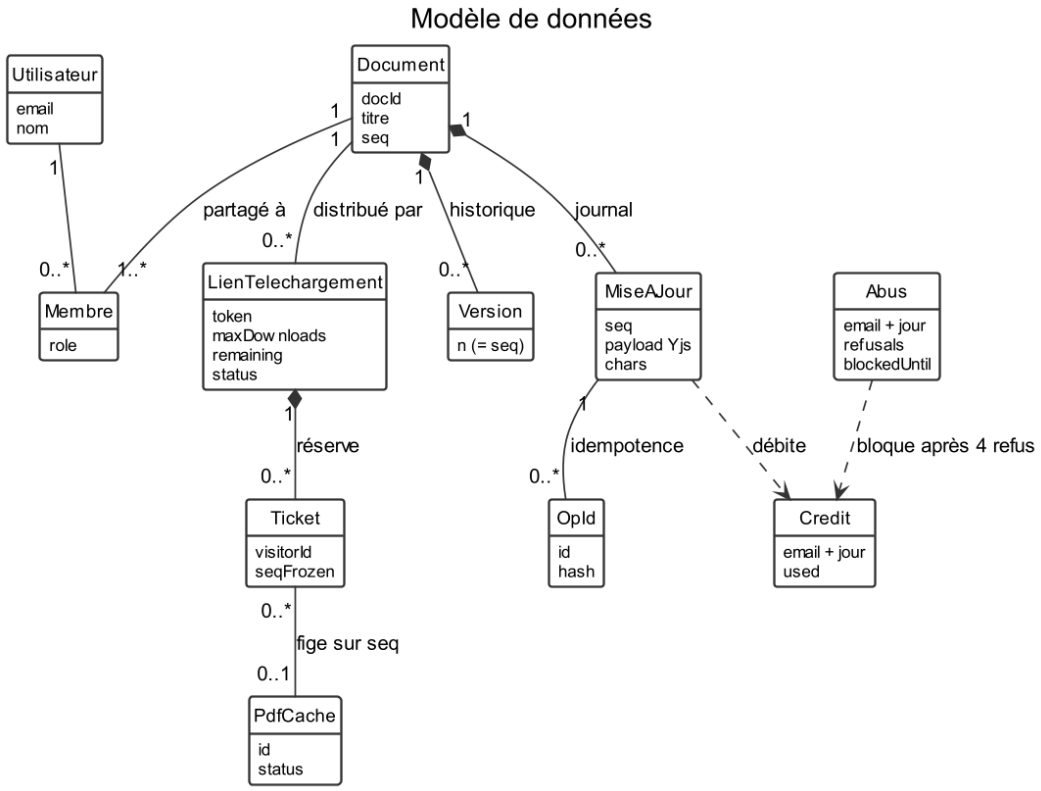


Figure 7 – Modèle du domaine : dix entités, trois portent une règle de concurrence ou de quota.

Trois points de conception méritent l'attention, parce qu'ils sont faciles à modéliser faux.

Le premier concerne les **versions**. Le numéro de version *est* le numéro de séquence du document au moment de l'enregistrement. Ce n'est pas deux champs mais un seul, utilisé deux fois. La conséquence pratique est agréable : la numérotation ne peut pas avoir de trous, puisqu'elle suit une séquence déjà garantie contiguë par le journal.

Le deuxième concerne l'**idempotence**. La table des marqueurs porte deux clés par mise à jour : l'identifiant d'opération fourni par le client, et l'empreinte du contenu. La seconde existe parce qu'un client qui se reconnecte peut renvoyer la même modification sous un autre identifiant ; sans elle, un renvoi débiterait le crédit deux fois.

Le troisième concerne le **cache de PDF**. Il porte un compteur de tentatives et une date de bail. Sans eux, deux visiteurs arrivant sur un PDF absent publieraient deux travaux de génération, et un PDF qui échoue serait redemandé indéfiniment.

Table	Clé	Rôle de conception
`Documents`	`docId`	séquence courante, dernier instantané, longueur du texte
`Members`	`docId` + `email`	appartenance ; index secondaire par adresse pour lister les documents d'un rédacteur
`Updates`	`docId` + `seq`	journal : une entrée par lot, avec l'update Yjs fusionné
`Versions`	`docId` + `n`	instantané consultable ; `n` est le numéro de séquence
`Links`	`token`	quota : `remaining` et statut
`Tickets`	`token` + `visitorId`	exemplaire réservé et numéro figé
`PdfCache`	`docId#seq`	état de génération, verrou, tentatives
`Credits`	`email#jour`	compteur consommé et plafond
`Abus`	`email#jour`	refus du jour et terme du blocage
`OpIds`	`id`	idempotence : identifiant d'opération et empreinte

Table 6 – Modèle de données : dix tables, une clé et un rôle par table.

4.4 Diagrammes d'états-transitions

Trois objets ont un cycle de vie qui porte une règle : le lien, le visiteur, et le crédit.

4.4.1 États du lien

Un lien naît actif avec son quota, le consomme, et devient épuisé. La transition est *monotone* : un lien ne redevient jamais actif. Point important pour la lecture du diagramme : l'état épuisé **n'est pas terminal**. Le lien reste consultable indéfiniment et répond « épuisé », ce qui est précisément ce que demande C6 — plus aucun travail inutile.

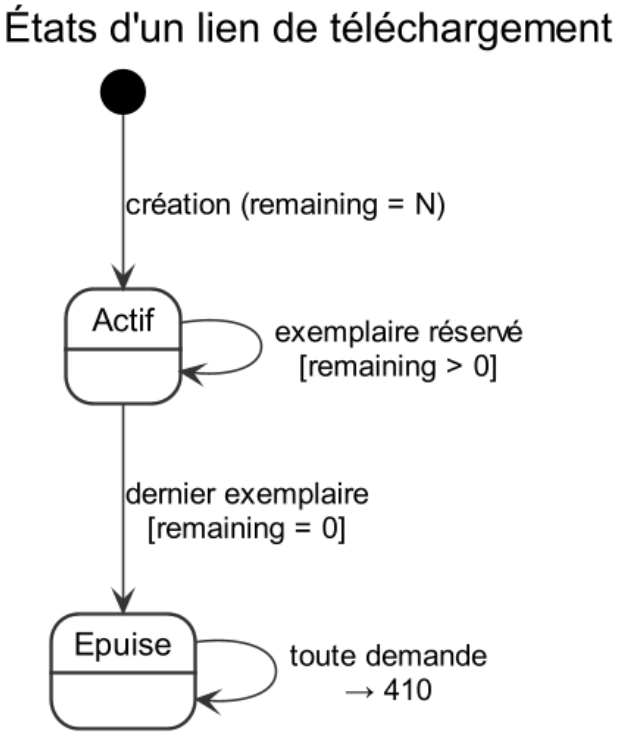


Figure 8 – États d'un lien de téléchargement.

4.4.2 États du visiteur

Le visiteur suit un chemin qui dépend de deux choses indépendantes : le quota du lien et l'état du fichier. C'est ce qui produit trois réponses distinctes là où on attendrait une seule : prêt, patienter, épuisé.

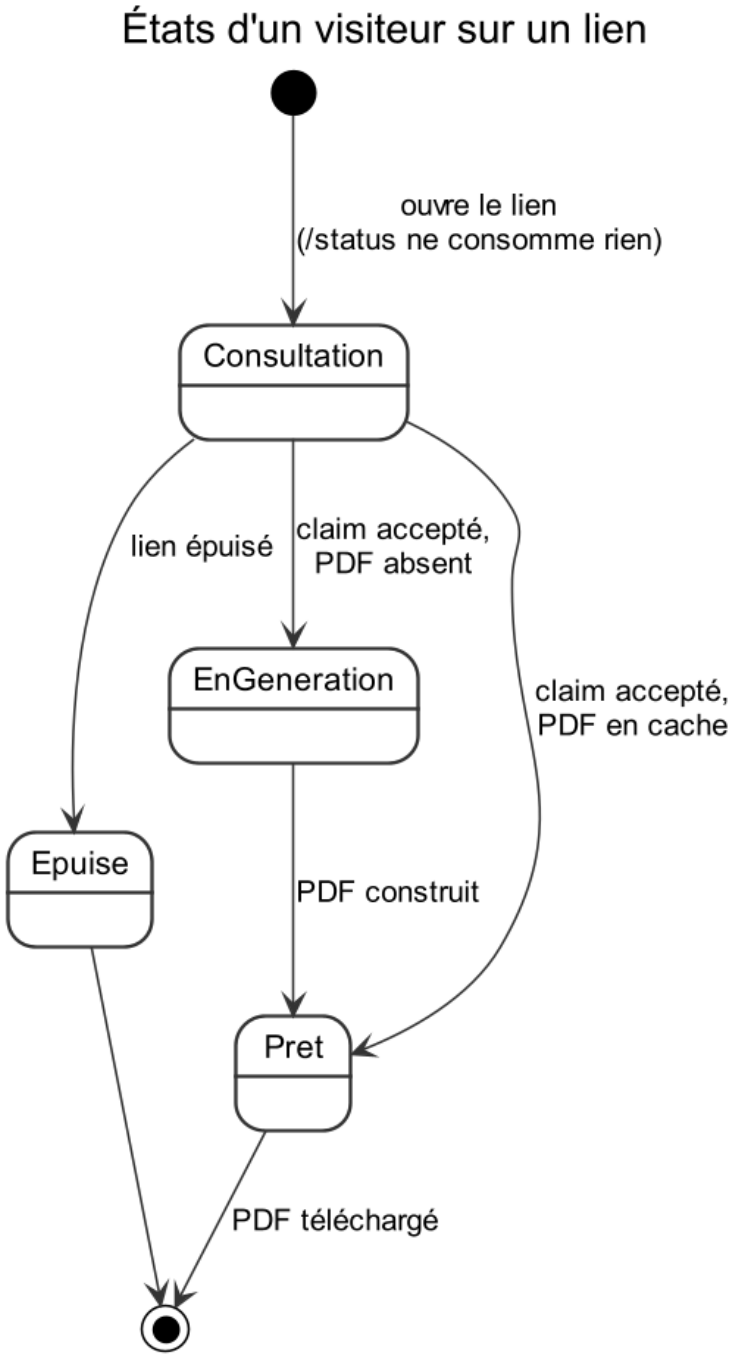


Figure 9 – États d'un visiteur sur un lien.

Un point mérite d'être souligné : une fois l'exemplaire réservé, il est *acquis*. L'URL présignée expire au bout de soixante secondes, mais le visiteur peut en redemander une indéfiniment. Rien de ce qu'il fait ensuite ne consomme d'exemplaire supplémentaire.

4.4.3 États du crédit

Le crédit a trois états, et le troisième est celui qu'on oublie : un utilisateur peut être *bloqué alors qu'il lui reste du crédit*. Le blocage est posé par quatre refus dans la journée ; il ne s'efface pas parce que le compteur a été remis à zéro.

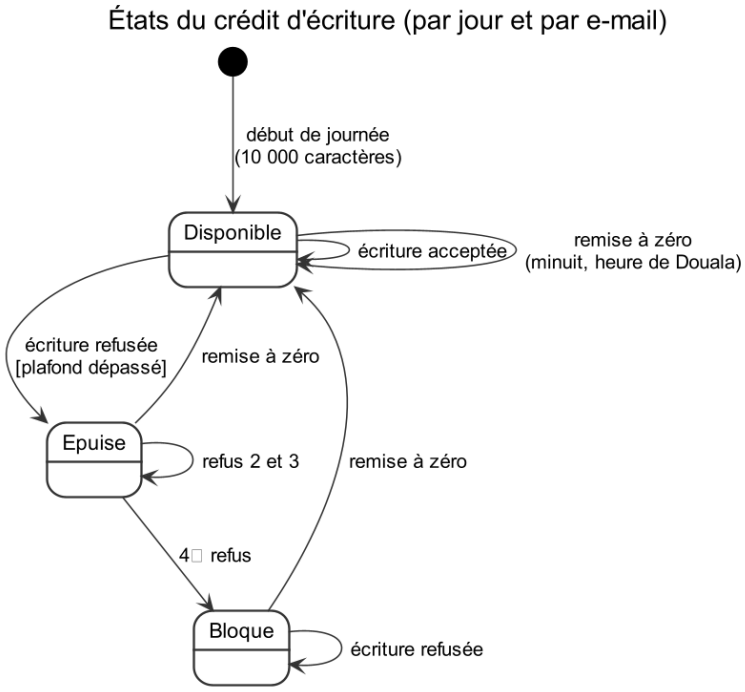


Figure 10 – États du crédit d'écriture, par jour et par adresse.

La remise à zéro mérite une explication, parce qu'elle a été source de confusion. Ce n'est pas un événement, c'est un changement de clé : le compteur est indexé par adresse et par jour. Deux journées différentes sont deux lignes différentes, sans qu'aucune écriture ne soit nécessaire.

Hypothèse d'interprétation. Le « jour » est celui de Douala, pas le jour UTC. C'est un écart avec l'hypothèse initiale du plan. Un utilisateur lisait « recharge à 01 :00 » et ne comprenait pas : minuit UTC tombe à une heure du matin sur place. Le décalage est constant — le Cameroun n'applique pas d'heure d'été — ce qui rend l'arithmétique exacte toute l'année. Le terme du blocage est par ailleurs *recalculé* à chaque lecture et non relu en base, ce qui a corrigé les lignes écrites sous l'ancienne règle sans aucune migration.

4.5 Diagrammes de séquence système

Trois parcours structurent l'application : réserver puis télécharger, éditer, restaurer.

4.5.1 Réserver et télécharger

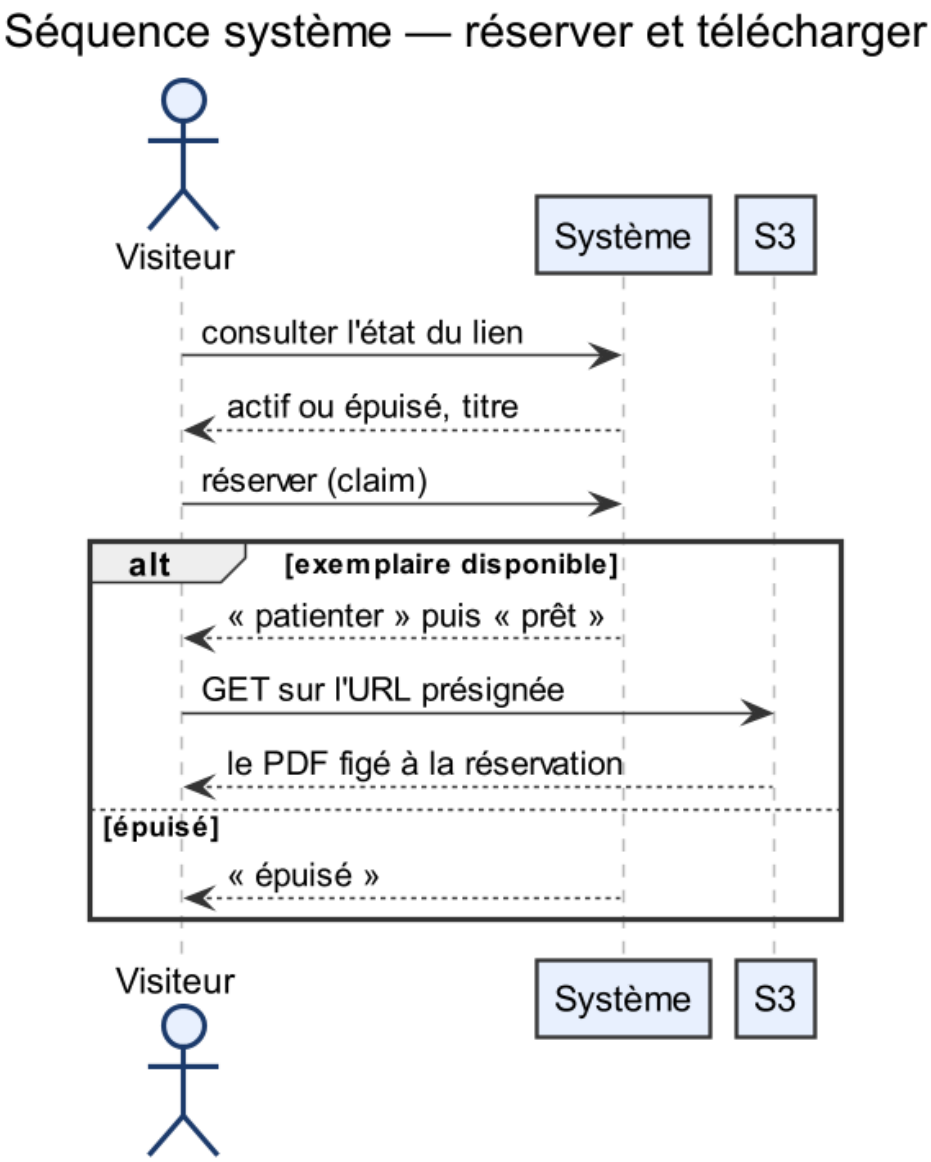


Figure 11 – Séquence système : réserver puis télécharger un exemplaire.

Deux précisions que le diagramme ne peut pas montrer. Consulter l'état du lien ne consomme rien : c'est une exigence de conception, sans quoi un aperçu de lien dans une messagerie ou un robot d'indexation épuiserait le quota. Et l'interrogation d'attente porte sur le *fichier*, pas sur l'état du lien : l'état du lien ne dit rien de l'avancement de la génération.

4.5.2 Éditer un document

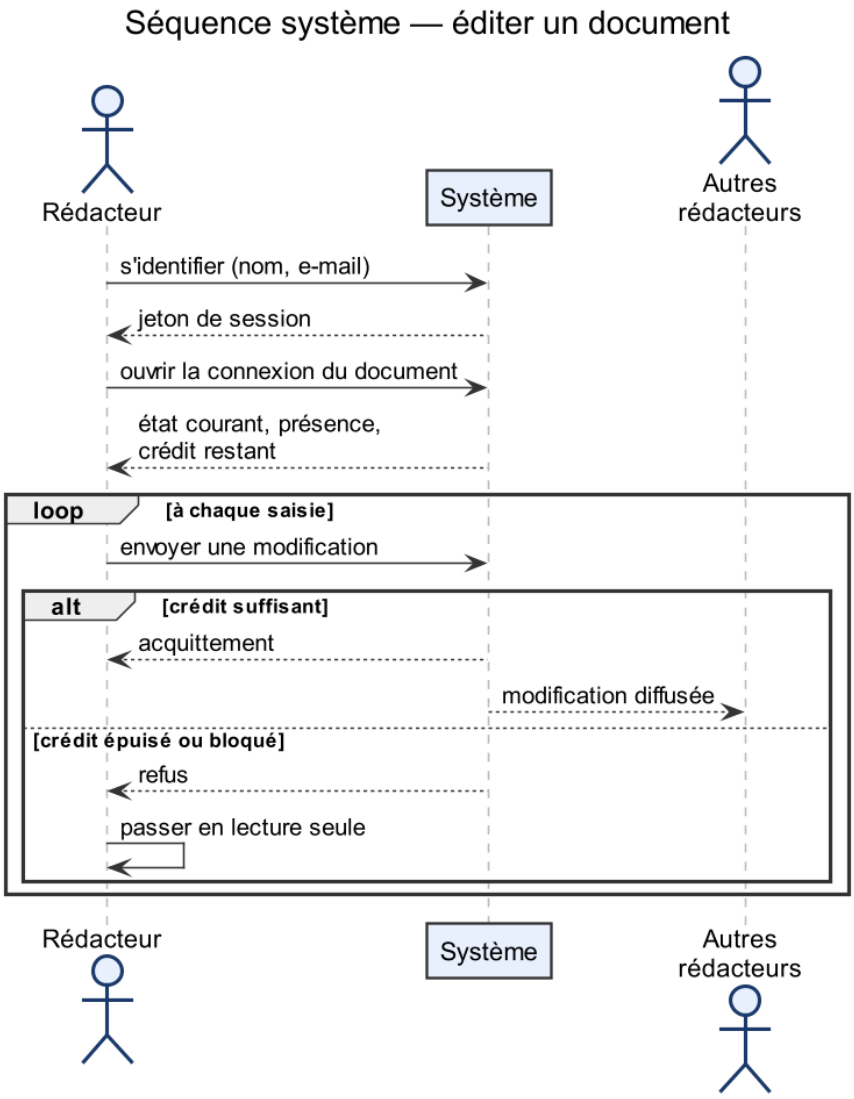


Figure 12 – Séquence système : éditer un document en collaboration.

Le comportement en cas de refus mérite d'être explicite, parce qu'il est contre-intuitif. À la réception d'un refus de crédit, le client vide sa file locale et reconstruit le document depuis le serveur. Ce n'est pas une resynchronisation : le refus n'a rien modifié. Le client reconstruit parce que les frappes déjà saisies ne seront *jamais* enregistrées, et qu'il faut le montrer sans ambiguïté plutôt que laisser un état intermédiaire trompeur.

4.5.3 Restaurer une version

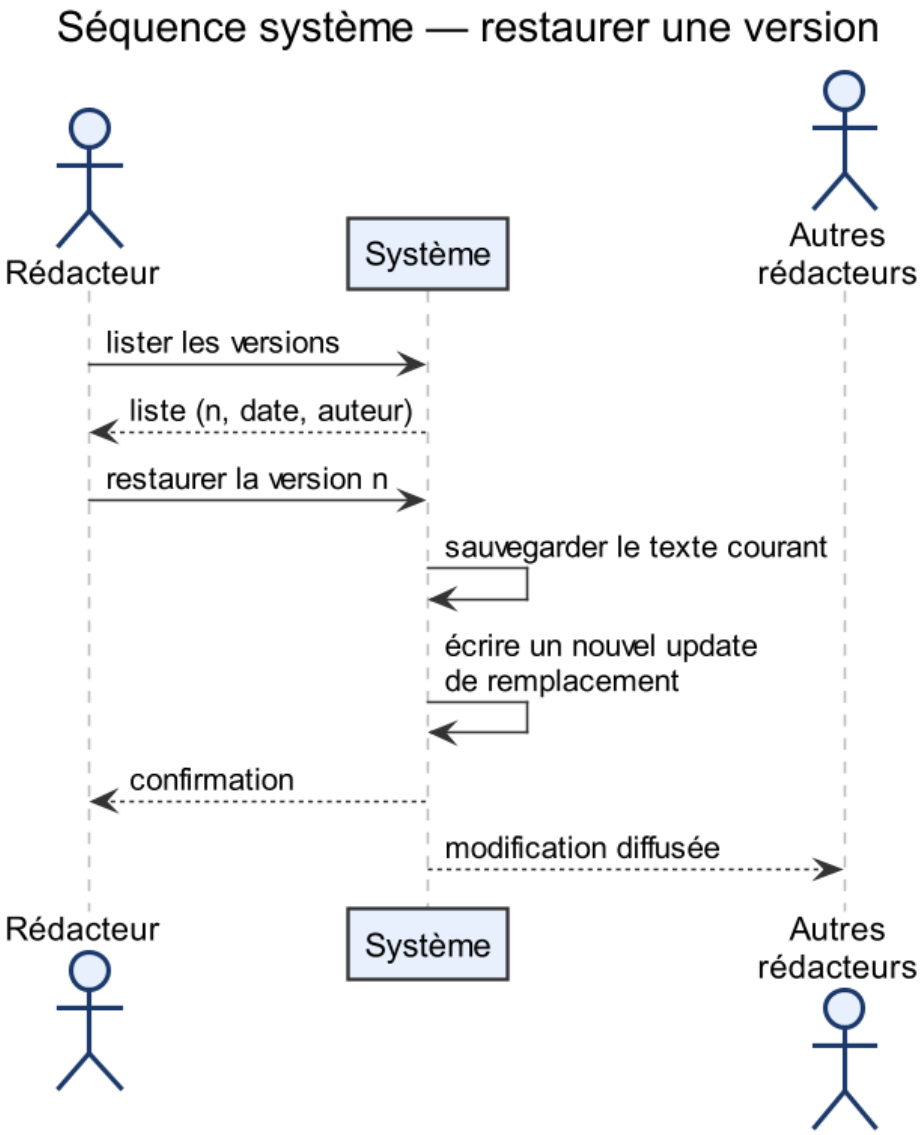


Figure 13 – Séquence système : restaurer une version.

La restauration est une écriture, pas un retour en arrière. Trois propriétés en découlent, et toutes les trois sont voulues : la compatibilité avec le CRDT, donc la convergence des autres clients ; la conservation de l'historique, puisque le texte écrasé est d'abord sauvegardé comme version ; la gratuité, puisqu'aucun caractère n'est facturé et que l'opération franchit un blocage. Cette dernière propriété est un choix : sans elle, un utilisateur ayant épuisé son crédit serait enfermé sans recours.

4.6 Conception par contrainte

Cette section reprend les six contraintes une par une, et indique pour chacune *où* elle se tient dans la conception, et *comment* elle est vérifiée. C'est la lecture la plus utile du dossier : elle relie chaque exigence à un mécanisme et à une preuve.

4.6.1 C1 — exactement N téléchargements

Où. Dans une seule transaction conditionnelle sur la table des liens et la table des tickets. Le décrément n'a lieu que si le compteur est positif et le lien actif ; le ticket n'est créé que si ce visiteur n'en a pas.

Ce qui rend la faute possible, et comment elle est évitée. Quatre pièges ont été identifiés :

1. chercher le ticket existant *après* le court-circuit « épuisé » : un visiteur déjà servi recevrait alors un refus. L'ordre est inversé exprès ;
2. compter l'exemplaire au téléchargement du fichier et non à la réservation : un visiteur qui recharge la page consommerait deux fois. L'exemplaire est consommé au claim ;
3. poser la transition « épuisé » dans la même écriture que le décrément : cinquante claims simultanés tenteraient alors cinquante fois la même transition. Elle est séparée et conditionnelle ;
4. croire que la garde en mémoire décide du quota. Elle ne fait qu'*alléger* la base sous le pic ; un visiteur qu'elle écarte reçoit « patienter », jamais « épuisé ».

Preuve. Onze tests unitaires, dont un test de concurrence : cinquante claims simultanés pour vingt exemplaires donnent exactement vingt réservations. En charge, 15 000 visiteurs sur un lien de 500 exemplaires ont produit *exactement* 500 réservations, sans une seule erreur serveur.

4.6.2 C2 — aucune modification acceptée perdue

Où. Dans l'écrivain unique par document et dans l'ordre acquittement-après-écriture. Un lot est écrit en une transaction atomique, puis diffusé en un seul message. Un acquittement ne part qu'après validation de la transaction.

Ce qui rend la faute possible, et comment elle est évitée. Trois risques :

1. acquitter avant d'écrire : une panne entre les deux perd la modification sans que personne ne le sache. L'acquittement est émis après ;
2. renvoyer une modification sous un nouvel identifiant après une coupure : elle serait écrite deux fois. Le client renvoie le *même* identifiant, et le serveur déduplique aussi par empreinte du contenu ;
3. mesurer la convergence avec un outil qui ne peut pas la mesurer. Un générateur de charge compte des acquittements, il ne fait pas tourner un client CRDT.

Preuve. Deux preuves distinctes, et c'est le point important. Le générateur de charge vérifie que chaque modification émise reçoit son acquittement. Un second outil, écrit en Node avec un vrai client CRDT, reconstruit le document depuis le journal, vérifie que les numéros de séquence sont contigus, et compare deux reconstructions indépendantes par empreinte. Ce second outil a été validé dans les deux sens : il conclut à la convergence sur un journal sain, et à l'échec dès qu'un numéro manque. Un détecteur qu'on n'a jamais vu détecter ne prouve rien.

4.6.3 C3 — document tel qu'il est à T_1

Où. Au moment du claim : le dernier numéro de séquence réellement écrit est figé dans le ticket. Le PDF est ensuite construit depuis le journal jusqu'à ce numéro.

Ce qui rend la faute possible. Le PDF pourrait être généré à la demande, au moment du téléchargement : deux visiteurs d'une même rafale recevraient alors des documents différents, ce qui viole l'exigence de complétude et de cohérence. Il pourrait aussi être régénéré à chaque visite, ce qui multiplierait le travail par le nombre de visiteurs au lieu du nombre de versions.

Preuve. Le contenu du PDF est déterminé par le numéro figé, et la clé de cache inclut ce numéro : une version, un fichier. Un verrou avec bail empêche deux générations simultanées pour la même version.

4.6.4 C4 — jamais plus que son crédit

Où. Dans un compteur par adresse et par jour, débité dans la même transaction que l'écriture au journal. Le comptage est effectué sur le binaire Yjs, côté serveur : le client ne déclare pas ce qu'il a écrit.

Ce qui rend la faute possible, et comment elle est évitée.

1. compter côté client : un client modifié déclarerait zéro. Le comptage est serveur, en décodant la mise à jour ;
2. ne pas facturer les suppressions : écrire puis effacer serait gratuit, et le plafond contournable en trois gestes. Les suppressions sont facturées ;
3. compter par connexion plutôt que par adresse : cinq onglets donneraient cinq quotas. La clé est l'adresse ;
4. inscrire le plafond comme condition de la transaction : DynamoDB refuse l'arithmétique entre valeurs. C'est une lecture préalable.

Preuve. Seize tests unitaires, dont l'acceptation exacte jusqu'au plafond et pas un caractère de plus, le partage du compteur entre plusieurs connexions, la déduplication par identifiant et par empreinte, et le blocage au quatrième refus.

4.6.5 C5 — 15 000 clics et 300 rédacteurs

Où. À quatre endroits complémentaires : le micro-cache du serveur frontal, la garde en mémoire sur les réservations, la diffusion par lots, et le dimensionnement de l'instance.

Ce qui rend la faute possible. Chaque mécanisme a un mode de défaillance propre, et c'est leur combinaison qui tient :

— sans micro-cache, 15 000 lectures d'état frappent la base et la font échouer en limitation de débit ;

— sans garde en mémoire, autant d'écritures conditionnelles simultanées produisent le même effet ;

— sans diffusion par lots, un message par frappe multiplie la charge réseau par le nombre de rédacteurs ;

— sans dimensionnement mesuré, une instance trop petite s'effondre précisément pendant le pic qu'on mesure.

Preuve. Le scénario mixte fait tourner visiteurs, rédacteurs et sondes d'état en même temps. Les mesures relevées sont la latence de réservation, la latence d'état, la latence d'acquittement, et le nombre d'erreurs serveur — celui-ci doit rester à zéro, y compris pour le parcours visiteur.

Un piège de mesure mérite d'être signalé : une réponse « épuisé » est un code 410, que les outils de charge comptent comme une requête échouée. Un seuil sur ce taux d'échec est donc inatteignable par construction — 84 % mesurés sans la moindre panne. Le critère utile est un compteur dédié aux erreurs serveur.

4.6.6 C6 — dernier exemplaire : prévenir vite, arrêter le travail

Où. Dans la monotonie de l'état épuisé et dans son cache. Une fois le lien épuisé, la réponse est immédiate, ne touche pas la base, et le serveur frontal la conserve longuement. Le créateur du lien reçoit un courriel unique.

Ce qui rend la faute possible. Deux choses : recalculer l'état à chaque visite, ce qui revient à travailler pour ne rien dire ; et envoyer le courriel plusieurs fois, une par visiteur qui constate l'épuisement. Le courriel est donc protégé par un verrou conditionnel : seul le claim qui pose la transition déclenche l'envoi.

Preuve. L'état épuisé est posé une seule fois, et la notification est idempotente : neuf tests unitaires couvrent l'envoi au seul dernier exemplaire, l'absence de second envoi, la panne du service de courriel sans impact pour le visiteur, et le relâchement du verrou en cas d'échec — mieux vaut un doublon rare qu'un créateur jamais prévenu.

#	Mécanisme	Preuve exécutable
C1	transaction conditionnelle, ordre des vérifications, transition séparée	11 tests dont concurrence 50/20 ; en charge, 500 réservations exactes
C2	écrivain unique, acquittement après écriture, déduplication par empreinte	acquittements sous charge ; contiguïté du journal et double reconstruction
C3	numéro figé au claim, une génération par version, verrou avec bail	clé de cache par version ; un seul travail par version
C4	comptage serveur sur le binaire, suppressions facturées, clé par adresse	16 tests : plafond exact, multi-connexions, blocage au 4e refus
C5	micro-cache, garde mémoire, diffusion par lots, dimensionnement mesuré	scénario mixte : visiteurs, rédacteurs et sondes simultanés
C6	état épuisé monotone et mis en cache, notification idempotente	9 tests : envoi unique, panne sans impact, verrou relâché

Table 7 – Synthèse : une contrainte, un mécanisme, une preuve.

5 Conclusion

5.1 Ce qui est tenu, et comment on le sait

Deux contraintes étaient éliminatoires. Elles sont traitées par des mécanismes différents, et vérifiées par des preuves différentes — ce qui est précisément pourquoi elles méritent d'être distinguées.

C1 est prouvée en charge, en production. Un lien de 500 exemplaires a reçu 16 753 requêtes dont 500 réservations exactement, sans une seule erreur serveur, avec un pic à 200 requêtes par seconde. Le compteur final est à zéro et l'état du lien est « épuisé ». Ce résultat n'est pas une moyenne : la contrainte est « exactement N », et l'écart mesuré est nul.

C2 est prouvée en deux temps. Sous charge, chaque modification émise a reçu son acquittement : aucun message manquant. Mais un acquittement ne prouve pas la convergence du contenu, et un outil de charge ne peut pas le faire. Un second outil, avec un vrai client CRDT, reconstruit le document depuis le journal, vérifie que les numéros de séquence sont contigus, et compare deux reconstructions indépendantes. Il conclut à la convergence, et il conclut à l'échec dès qu'un numéro manque.

Les quatre autres contraintes sont couvertes par des tests unitaires. Le quota de crédit, la version figée du PDF, le comportement sous pic et la notification du dernier exemplaire ont chacun leur jeu de tests, sans aucun appel à un service distant.

5.2 Ce qui reste à mesurer

Le dossier doit être honnête sur ce point.

Le nombre de rédacteurs simultanés effectivement tenu reste à établir par paliers successifs, et le résultat doit être rapporté tel quel, même inférieur à la cible de 300. Une cible non atteinte et annoncée vaut mieux qu'un chiffre flatteur et invérifiable.

Le comportement après un arrêt brutal du processus en pleine écriture dispose d'un scénario dédié, qui tue le processus et vérifie que le journal a progressé sans trou après redémarrage. Ce scénario est écrit, son exécution reste à consigner.

Les métriques système — occupation processeur, mémoire de pointe, solde de crédits de processeur — ne sont pas observables depuis l'outil de charge. Elles se relèvent en parallèle sur la supervision du fournisseur, et doivent accompagner les résultats de charge pour que le dimensionnement soit justifié plutôt qu'affirmé.

5.3 Ce que ce projet a réellement appris

Trois leçons, dans l'ordre de ce qu'elles ont coûté.

Première leçon : un faux service doit imiter les refus, pas seulement les succès. Un double en mémoire de la base évaluait lui-même les expressions de condition. Il était donc plus permissif que le vrai service. Résultat : quarante-deux tests verts, et aucune écriture possible en production, parce que le service réel refuse l'arithmétique entre deux valeurs dans une condition. Le même défaut s'est reproduit avec les mots réservés. La correction n'a pas été de réparer le code, mais de rendre le double *plus strict* : il rejette désormais ce que le service rejette.

Deuxième leçon : une compilation réussie ne prouve rien. Les douze diagrammes de conception compilaient sans erreur, et plusieurs étaient faux : un code 200 là où le service répond 202, une redirection qui n'existe pas, un numéro de version compté deux fois. L'outil de diagrammes écrit ses erreurs dans l'image et renvoie un succès. Il a fallu relire chaque source contre le code et regarder chaque rendu.

Troisième leçon : un test qui ne peut pas échouer ne protège rien. Un seuil de taux d'échec posé sur un parcours qui renvoie légitimement des refus est inatteignable par construction. Un outil de vérification qui interroge une route dont la réponse n'a pas la forme attendue ne vérifie rien. À chaque fois, la question utile n'est pas « le test passe-t-il ? » mais « ai-je déjà vu ce test échouer ? ». C'est pour cette raison que les trois détecteurs de ce projet ont été éprouvés en provoquant la panne qu'ils prétendent détecter.

5.4 Perspectives

Trois évolutions seraient utiles sans être nécessaires.

Le chiffrement du trafic suppose un nom de domaine : c'est la limite la plus visible du déploiement actuel, et la plus simple à lever.

La haute disponibilité suppose de traiter l'écrivain unique autrement : élection d'un responsable par document, ou externalisation de l'état. C'est un chantier d'architecture à part entière, pas un réglage.

Le palier de rédacteurs, enfin, se mesure avant de s'optimiser : le résultat réel doit guider la suite, plutôt qu'une optimisation supposée.

5.5 Mot de la fin

Le sujet prévenait : une application simple dont chaque choix est compris vaut mieux qu'une application complexe qu'on ne sait pas expliquer. Ce dossier a suivi cette consigne à la lettre, y compris dans ce qu'elle a de désagréable : les limites sont énoncées, les hypothèses sont marquées

comme telles, et les écarts entre le plan et l'implémentation sont rapportés plutôt que lissés. C'est la partie du travail qui a le plus servi, parce que c'est celle qui a révélé les défauts que les tests ne voyaient pas.